CS65: Lab 6

Assigned Thursday, October 24
Due Tuesday, October 30

1 Introduction

In this lab, you will implement the Forward algorithm, the Backward algorithm, the Viterbi algorithm, and the Forward-Backward algorithm for Hidden Markov Models. The goal of the lab is to reinforce what each of these HMM algorithms does, and how they accomplish it.

There are a few significant typos in the text. On the textbook's website there is a list of errata. Choose the version of the textbook you have and scroll down to Chapter 6:
`http://www.cs.colorado.edu/~martin/SLP/errata-version.html` In this handout, I have reproduced equations and their descriptions directly from the book, correcting errors where necessary. You can use this handout as a quick guide to the equations in the text, but this is not a substitute for reading the text and understanding the algorithms.

Some implementation notes:

1. When implementing these algorithms, be sure to keep your indexes correct. A number of these dynamic programming algorithms are written to start at index 1, not index 0. This means that in many cases you'll need to allocate extra space in your lists and waste the zero$^{\text{th}}$ position.
2. The textbook uses N to represent the number of states *excluding* the start and final state.
3. Be sure you read the errata page mentioned above when referring to the examples in the textbook.

You will probably want to download Jason Eisner's Excel spreadsheets that demonstrate the Forward-Backward algorithm (`http://www.cs.jhu.edu/~jason/papers/eisner.hmm.xls`) and the Viterbi algorithm (`http://www.cs.jhu.edu/~jason/papers/eisner.hmm-viterbi.xls`).

2 WARNING! Avoid Very Unhappy Debugging Land!

The actual code that you need to implement for this lab is not incredibly easy, but it should be more straightforward than you think it's going to be. That said, it is extremely easy to make index errors and other small errors that are close-to-impossible to find unless you **test often**. If you write a method, be sure you test it. Then test it some more. Each part is quite dependent on the next, so errors will propagate and you will be in very unhappy debugging land. It's a terrible place to visit and you don't want to go.

3 Starting point code

I have provided you with starting point code that creates a class called HMM which is initialized by passing it Q (the list of states), V (the vocabulary), A (the two-dimensional matrix of transition probabilities), and B (the two-dimensional matrix of output probabilities). The code also provides functions to read in and write out the specifications for an HMM, a partial implementation of the forward-backward algorithm, as well as some convenience functions for creating 2- and 3-dimensional matrices. The HMM class also has stubs for methods that compute the forward probability α, the backward probability β, methods that compute γ and ξ as part of the E-step, methods that compute $\hat{A}$ and $\hat{B}$ as part of the M-step, a method that computes the Viterbi matrix (and its backpointers), and a method that, given the backpointers matrix, tells you what the most likely sequence is.

Be sure you read through the code to make sure you understand what is and, more importantly, what is not implemented before you proceed.

You can run the program as follows:

```
python hmm.py <hmmfile> <sequencefile> <likelihood|decode|em:iterations>
```

For example, to compute the likelihood of the sequence shown in the Excel spreadsheet using the ice cream HMM, you would type:

```
python hmm.py icecream.hmm eisnerseq.txt likelihood
```

3.1 HMM file format

Formally, we would represent the ice cream HMM in Jason Eisner's spreadsheet as follows:

```
Q = ['START', 'COLD', 'HOT', 'END']
V = ['1', '2', '3']   #output vocabulary
A = [[0.0, 0.5, 0.5, 0.0],
     [0.0, 0.8, 0.1, 0.1],
     [0.0, 0.1, 0.8, 0.1],
     [0.0, 0.0, 0.0, 1.0]]
B = [[0.0, 0.0, 0.0],
     [0.7, 0.2, 0.1],
     [0.1, 0.2, 0.7],
     [0.0, 0.0, 0.0]]
```

The starting point code can read and write HMM to a text file. Below left is the specification for the file format. Below right is the ice cream HMM shown above encoded using the file format. This can also be found in `labs/04/icecream.hmm`.

```
<number of states>
<name of start state>
<name of state 1>
...
<name of final state>
<number of vocabulary items>
<vocabulary item 1>
<vocabulary item 2>
...
A00 A01 A02 ...
A10 A11 A12 ...
...
B00 B01 B02 ...
B10 B11 B12 ...
...
# Empty lines and comments beginning
# with a # sign are ignored.
```

```
4       #number of states
'START'
'COLD'
'HOT'
'END'
3       #size of vocabulary
'1'
'2'
'3'
#A matrix
0.0 0.5 0.5 0.0
0.0 0.8 0.1 0.1
0.0 0.1 0.8 0.1
0.0 0.0 0.0 1.0
#B matrix
0.0 0.0 0.0
0.7 0.2 0.1
0.1 0.2 0.7
0.0 0.0 0.0
```

3.2 Output sequence file format

The program is set up to read an output sequence from a text file. You should put one output symbol per line. Be sure your output symbols match those in your HMM's vocabulary. The example shown below is provided in `labs/04/sequence.txt`:

```
'3'
'1'
'3'
'2'
'2'
'3'
```

4 Hidden Markov Model definition

Here is the formal definition of a Hidden Markov Model given in the textbook.

$Q = q_1 q_2 ... q_N$	a set of N **states**
$A = a_{11} a_{12} ... a_{n1} ... a_{nn}$	a **transition probability matrix** A, each a_{ij} representing the probability of moving from state i to state j, such that $\sum_{j=1}^{n} a_{ij} = 1 \quad \forall i$
$O = o_1, o_2, ..., o_T$	a sequence of T **observations**, each one drawn from a vocabulary $V = v_1, v_2, ..., v_V$.
$B = b_i(o_t)$	a sequence of **observation likelihoods**, also called **emission probabilities**, expressing the probability of an observation o_t being generated from a state i.
q_0, q_F	a special **start state** and **end (final) state** that are not associated with observations, together with transition probabilities $a_{01} a_{02} ... a_{0n}$ out of the start state and $a_{1F} a_{2F} a_{3F}$ into the end state

5 The forward algorithm

$\alpha_t(j)$ represents the probability of being in state j after seeing the first t observations, given the HMM λ:

$$\alpha_t(j) = P(o_1, o_2, ..., o_t, q_t = j | \lambda)$$

The recursive dynamic programming algorithm for computing $\alpha_t(j)$ is:

1. Initialization

$$\alpha_1(j) = a_{0j} b_j(o_1) \qquad 1 \leq j \leq N$$

2. Recursion

$$\alpha_t(j) = \sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} b_j(o_t) \qquad 1 \leq j \leq N, 1 < t \leq T$$

3. Termination

$$P(O|\lambda) = \alpha_T(q_F) = \sum_{i=1}^{N} \alpha_T(i) a_{iF}$$

NOTE: Just because the above mathematical definition is recursive does **not** mean that you should be writing recursive functions: I am expecting that you will implement the equations above using a series of nested for-loops.

1. Implement the Forward algorithm. Refer to the equations above and the pseudo-code in Figure 6.9 (p.183). Test your implementation using the HMM in `icecream.hmm` by computing the probability of each of the following observation sequences.

   (a) 313 (the example we did in class, found in `313.txt`)

   (b) 33112113331 (found in `sequenceA.txt`)

   (c) 33113123331 (found in `sequenceB.txt`)

   (d) The sequence from the Excel spreadsheet (found in `eisnerseq.txt`)

   You can verify that your implementation is correct using pencil and paper (for the entire first sequence, and by examining the α values for first few steps of the next two sequences), and by comparing your answer for the final sequence to that found in Jason Eisner's `eisner.hmm.xls` spreadsheet (in any of the values in column `I`).

6 The backward algorithm

$\beta_t(i)$ represents the probability of seeing the observations from $t+1$ to the end, given that we are in state i at time t, given λ.

$$\beta_t(i) = P(o_{t+1}, o_{t+2}, ..., o_T | q_t = i, \lambda)$$

The recursive dynamic programming algorithm for computing $\beta_t(i)$ is:

1. Initialization

$$\beta_T(i) = a_{iF} \qquad 1 \leq i \leq N$$

2. Recursion

$$\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(o_{t+1}) \beta_{t+1}(j) \qquad 1 \leq i \leq N, 1 \leq t < T$$

3. Termination

$$P(O|\lambda) = \alpha_T(q_F) = \beta_1(q_0) = \sum_{j=1}^{N} a_{0j} b_j(o_1) \beta_1(j)$$

2. Implement the Backward algorithm, referring to the equations above. Test your implementation using `icecream.hmm` by computing the probability of the same observation sequences from the previous question. You can verify your implementation by ensuring you get the same likelihood as you did in the previous question for each sequence. You may want to add a new command-line argument (e.g. `backward`) that prints out the likelihood as computed by the backward algorithm.

7 The Viterbi algorithm

$v_t(j)$ represents the probability that the HMM is in state j after seeing the first t observations and passing through the most probable state sequence $q_0, q_1, ..., q_{t-1}$, given λ.

$$v_t(j) = \max_{q_0, q_1, ..., q_{t-1}} P(q_0, q_1, ..., q_{t-1}, o_1, o_2, ..., o_t, q_t = j | \lambda)$$

At the termination of the Viterbi algorithm, $v_T(q_F)$ represents the probability that the HMM is in the final state after observing the entire sequence, having passed through the most likely sequence of states.

In order to recover the most likely sequence of states, the Viterbi algorithm also maintains a backtrace, $bt_t(j)$, that points to the most likely state the HMM was in at $t-1$, given that HMM is currently in state j after seeing the first t observations.

The recursive dynamic programming algorithm for computing $v_t(j)$ and $bt_t(j)$ is:

1. Initialization

$$v_1(j) = a_{0j} b_j(o_1) \qquad 1 \leq j \leq N$$
$$bt_1(j) = 0$$

2. Recursion

$$v_t(j) = \max_{i=1}^{N} v_{t-1}(i) a_{ij} b_j(o_t) \qquad 1 \leq j \leq N, 1 < t \leq T$$
$$bt_t(j) = \operatorname*{argmax}_{i=1}^{N} v_{t-1}(i) a_{ij} b_j(o_t) \qquad 1 \leq j \leq N, 1 < t \leq T$$

3. Termination

$$P* = v_T(q_F) = \max_{i=1}^{N} v_T(i) a_{iF}$$
$$q_T* = bt_T(q_F) = \operatorname*{argmax}_{i=1}^{N} v_T(i) a_{iF}$$

Note that the values stored in the backpointers $bt_t(j)$ are integers that are referring to states in Q. In other words, if $bt_t(j) = k$, this means that the previous state was q_k. For example, in the initialization step, $bt_1(j) = 0$ indicates that at time 1, the previous state was q_0, the start state.

3. Implement the Viterbi algorithm, including the calculation of v and bt, and complete the `backtrace` method to find the most likely state sequence given bt. Test your implementation using the HMM in `icecream.hmm` by computing the probability of the same observation sequences from the previous two questions. You can verify your implementation is correct using pencil and paper (for the entire first sequence, and by examining the v values for the first few steps of the next two sequences), and by comparing the results to those found in Jason Eisner's `eisner.hmm-viterbi.xls` spreadsheet.

   Note that the spreadsheet uses a slightly different calculation method than the book presents, so compare your Viterbi values against columns `C` and `D` ($\mu(C)$ and $\mu(H)$). Once you have found the best path (using your `backtrace` method), you can compare this to the values in columns `J` and `K`. If column `J` is a 1 for time t, then your path should pass through state `'C'` at time t; if column `K` is a 1 for time t, then your path should pass through state `'H'` at time t. Exactly one of columns `J` and `K` will be 1 for any time t.

   You should be able to run your decoding implementation as follows:

```
python hmm.py icecream.hmm eisnerseq.txt decode
```

8 The forward-backward algorithm

The forward-backward algorithm is an example of an EM learning algorithm. As such, there is an E-step and an M-step. In the starting point code, I have completed the high-level implementation of the forward-backward algorithm (in the `forwardBackward` method), the E-step (in the `E` method), and the M-step (in the `M` method). However, in order for them to return the correct values, you must implement

the low-level implementation of both. In particular, the E-step requires calculation of γ and ξ which you will implement in `gammaCalc` and `xiCalc`.[1] Similarly, the M-step requires calculation of $\hat{a}$ and $\hat{b}$ which you will implement in `ahatCalc` and `bhatCalc`.

For the E-step, γ and ξ are calculated as follows:

$$\gamma_t(j) = \frac{\alpha_t(j)\beta_t(j)}{\alpha_T(q_F)} \qquad 1 \leq t \leq T, 1 \leq j \leq N$$

$$\xi_t(i,j) = \frac{\alpha_t(i)a_{ij}b_j(o_{t+1})\beta_{t+1}(j)}{\alpha_T(q_F)} \qquad 1 \leq t < T, 1 \leq i \leq N, 1 \leq j \leq N$$

For the M-step, there is a mistake in the textbook. As written, it incorrectly re-estimates the transition probability between the non-start/non-final states. Also, it fails to re-estimate the transition probabilities from the start state and to the final state. To fix those errors, calculate $\hat{a}$ as follows:

$$\begin{aligned}
\hat{a}_{0j} &= \gamma_1(j) && 1 \leq j \leq N \\
\hat{a}_{ij} &= \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T} \gamma_t(i)} && 1 \leq i \leq N, 1 \leq j \leq N \\
\hat{a}_{iF} &= \frac{\gamma_T(i)}{\sum_{t=1}^{T} \gamma_t(i)} && 1 \leq i \leq N
\end{aligned}$$

You can calculate $\hat{b}$ as described in the textbook:

$$\hat{b}_j(v_k) = \frac{\sum_{t=1 \text{ such that } O_t=v_k}^{T} \gamma_t(j)}{\sum_{t=1}^{T} \gamma_t(j)} \qquad 1 \leq j \leq N, 0 \leq k < |V|$$

CAREFUL: Two very important warnings here:

- When computing $\hat{a}$ and $\hat{b}$, remember that these are re-estimations for the values in A and B. Any value that you don't re-estimate (the transition probabilities out of the final state and the emission probabilities from both the start and final state) need to be copied from A and B or you will output incorrectly trained HMMs.
- When computing $\hat{b}$, note that the `readSeq` method of the HMM class converts the input sequence, which is comprised of strings, into integers. (See the comment in the `readSeq` method.) In the above equation, it asks you to compare `O[t]` with `V[k]`, but in your implementation, `O[t]` is an integer and `V[k]` is a string. What you really need to check is whether or not `O[t] == k`.

4. Implement the four methods described above (`gammaCalc`, `xiCalc`, `ahat`, and `bhat`). Run your implementation on all four sequences. You should be able to hand compute the first few steps of the shortest sequence.

   You should compare your results for the last sequence directly with those presented in the spreadsheet. It is a good exercise to try and figure out where the values you are calculating (α, β, ξ and γ) are stored in the Excel spreadsheet. Run EM for 10 iterations and be sure your 'final' values for A and B match those in the spreadsheet.

   You should be able to run your forward-backward implementation for 10 iterations as follows:

   ```
   python hmm.py icecream.hmm eisnerseq.txt em:10
   ```

[1] The E-step also requires that you have successfully calculated α in `forward` and β in `backward`, both which you should have completed previously in the lab.